Open Access | https://doi.org/10.59350/6mxqs-3p708

Mesh Variables and PETSc Vectors: Keeping Arrays in Sync

Louis Moresi [1] [1] Australian National University

Abstract

One of the less glamorous but important problems in a finite element framework is this: how does the user assign values to a field variable, and how does the framework ensure that PETSc sees those values correctly, in parallel, without the user needing to keep track themselves?

Keywords Underworld Code, development

In Underworld2, the answer was context managers. You would wrap every data access in a `with` block, and the framework synchronised the arrays on exit. It was safe, but verbose.

In Underworld3, you just write to the array. The synchronisation happens automatically.

```
# Underworld2 (old)
with mesh.access(temperature):
    temperature.data[...] = values

# Underworld3 (current)
temperature.array[...] = values
```

This post explains how we make that work.

1. The Problem

PETSc stores field data in distributed vectors. Each MPI rank owns a portion of the mesh and holds a *local vector* (`_lvec`) that includes ghost values from neighbouring ranks. The solver reads and writes these local vectors during assembly and solution.

The user wants to work with NumPy arrays. They want to set initial conditions, apply corrections, read solution values — all using familiar NumPy indexing. They do not want to know about local vectors, global vectors, ghost regions, or scatter operations.

The challenge is bridging these two worlds without introducing bugs. If the user modifies an array but PETSc doesn't see the change, the solver works with stale data. If PETSc rebuilds its internal data structures (because the mesh adapted or a new variable was added), the user's cached array might try to view points at freed memory locations.

2. NDArray_With_Callback: A Reactive NumPy Array

The core mechanism is `NDArray_With_Callback` — a NumPy ndarray subclass that fires a callback whenever its data is modified. When you write `temperature.data[0:10] = 300.0`, the array detects the assignment and triggers a synchronisation callback that copies the modified values into the PETSc local vector and scatters them to neighbouring ranks.

PUBLISHED Apr 03, 2026 ARTICLE UWTN 2026-004 VERSION 1.0.0 LICENCE CC-BY-4.0 DOI 10.59350/6mxqs-3p708

The user sees a NumPy array. Behind it, every write triggers:

1. Values are written into the PETSc local vector
2. A local-to-global scatter copies owned values to the global vector
3. A global-to-local scatter fills ghost regions from neighbouring ranks

After step 3, every rank has consistent data including ghost values. The solver can proceed safely.

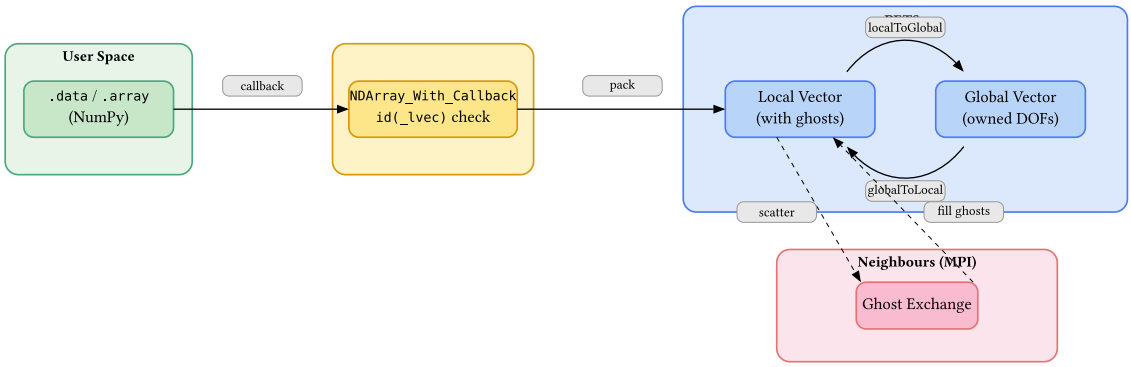

Figure 1: The data synchronisation pipeline: a write to .data or .array triggers a callback that packs values into the PETSc local vector, scatters to the global vector, and fills ghost regions from neighbouring MPI ranks.

3. The Self-Validating Cache

The `.data` property on a MeshVariable returns an `NDArray_With_Callback` view into the PETSc local vector. Creating this view is not free — it involves extracting the raw pointer from PETSc, wrapping it in NumPy, registering callbacks, and reshaping. So the variable caches it.

The danger is stale caches. PETSc can destroy and recreate its internal vectors when:

- A new MeshVariable is added to the mesh (triggers a DM rebuild)
- The mesh adapts (new topology, new vectors)

After either event, the old cached array view points at deallocated memory. Reading it returns garbage; writing to it corrupts the heap.

UW3 solves this with a single line of defence: on every `.data` or `.array` access, it checks whether `id(self._lvec)` matches the cached value. Python's `id()` returns the memory address of an object. If PETSc has replaced the local vector, the new object has a different `id`, the check fails, and the cache rebuilds automatically.

```
@property
def data(self):
    cache_valid = (
        self._canonical_data is not None
        and self._canonical_data_lvec_id == id(self._lvec)
    )

    if not cache_valid:
        self._canonical_data = self._create_canonical_data_array()
```

```
        self._canonical_data_lvec_id = id(self._lvec)

    return self._canonical_data
```

No code path needs to manually invalidate the cache. No flag to set, no method to call. The cache validates itself on every access. If the underlying vector changed, the view rebuilds. If it didn't, the cached view is returned immediately.

4. Batch Updates

Sometimes you need to update several variables together. Each individual write triggers a PETSc synchronisation — which involves MPI communication. If you are setting initial conditions on velocity, pressure, and temperature, that is three synchronisation rounds where one would suffice.

`uw.synchronised_array_update()` defers all callbacks until the context exits:

```
with uw.synchronised_array_update():
    velocity.array[...] = v_initial
    pressure.array[...] = p_initial
    temperature.array[...] = T_initial
# All three synchronise here, once
```

During the context, writes accumulate but callbacks are queued. On exit, all queued callbacks fire in order, and MPI barriers ensure all ranks stay in step.

5. Two Access Layers

MeshVariable exposes two properties for data access:

`.data` returns a flat `(N, num_components)` array. This is the internal format — what PETSc stores. It is always dimensionless (non-dimensionalised if units are active). Direct, fast, no conversion overhead.

`.array` returns a structured `(N, a, b)` array where the shape reflects the variable type: `(N, 1, 1)` for scalars, `(N, 1, dim)` for vectors, `(N, dim, dim)` for tensors. It handles unit conversion on read and write — you can assign values with physical units and they are non-dimensionalised before reaching PETSc.

For most user code, `.array` is a good choice. `.data` is there when you want to avoid the overhead of unit converstions or resizing (for example, copying from one array to another).

6. What the Solver Sees

The solvers themselves never use `.data` or `.array`. They access the PETSc vector directly via a `.vec` property that returns the raw `_lvec`. This is deliberate — the callback mechanism adds a thin layer of overhead that is irrelevant for user operations but would accumulate over millions of quadrature-point evaluations during assembly.

The division is clean: users work through `.array` (safe, synchronised, cached). Solvers work through `.vec` (direct, fast, PETSc-native). The two paths share the same underlying memory — the PETSc local vector — so there is no data duplication.

7. Why This Design

The context-manager approach in UW2 was safe but required discipline. Every data access had to be wrapped. Nested access for multiple variables was awkward. And the most common bug — forgetting the context manager — produced wrong results silently.

The callback approach eliminates an entire class of bugs. You cannot forget to synchronise because synchronisation is automatic. The self-validating cache eliminates another class — stale views after DM rebuilds. And batch updates via `synchronised_array_update()` give you the performance of explicit synchronisation when you need it.

The cost is one `id()` comparison per `.data` access and one callback dispatch per write. For user-level operations — setting initial conditions, post-processing solution fields, checkpointing — this is negligible. For solver-level operations — millions of quadrature evaluations — the direct `.vec` path bypasses it entirely.